STUDENT RECORD MANAGEMENT SYSTEM

A Menu-Driven Python Application

Course: Python Programming

Student: Kirabo Eria

Date: 30 June 2026

1. INTRODUCTION

The Student Record Management System is a menu-driven Python application designed to manage student records efficiently. The system demonstrates key programming concepts including:

- File I/O operations (CSV and JSON)
- Exception handling (try/except/finally)
- Custom exceptions
- Logging
- Input validation
- Modular programming

The system stores core student data in a CSV file and additional details in a JSON file, with all actions logged to a log file for auditing purposes.

2. PROGRAM DESIGN

2.1 System Architecture

The system follows a modular design with the following components:

Component	Description
CSV File (students.csv)	Stores core student data: reg_number, name, program, year

Component	Description
JSON File (students_details.json)	Stores additional details: address, contact, email, date_added
Log File (student_system.log)	Records all user actions and system errors
Main Program (student_management.py)	Contains all functions and menu system

2.2 Why CSV and JSON?

File Type	Purpose	Why Used
CSV	Core student data	Simple, spreadsheet-compatible, easy to view/edit
JSON	Additional details	Structured, supports nested data, better for complex fields

2.3 System Flow

The program follows this basic flow:

1. User launches the program
2. Files are initialized (created if they don't exist)
3. Main menu is displayed
4. User selects an option
5. Appropriate function is called
6. Data is read from/ written to files
7. Result is displayed to user
8. Action is logged
9. Return to main menu
10. Repeat until user chooses to exit

---

3. KEY FUNCTIONS

3.1 Function Overview

Function	Purpose
add_student()	Adds new student with auto-generated reg number and validation
view_all_students()	Displays all students in formatted table
view_student_details()	Shows complete details of one student
search_students()	Searches by name or registration number
update_student()	Updates both CSV and JSON data
delete_student()	Deletes student with confirmation
validate_reg_number()	Validates registration number format (S + 9 digits)
validate_name()	Validates name (letters and spaces only)
validate_email()	Validates email format
load_students()	Loads data from CSV
save_students()	Saves data to CSV
load_json_details()	Loads data from JSON
save_json_details()	Saves data to JSON

3.2 Function Flow

Add Student Flow:

1. Auto-generate registration number (S100000001, S100000002, ...)
2. Get user input (name, program, year)
3. Get additional details (address, contact, email)
4. Validate all inputs
5. Check if registration number exists

6. Save to CSV
7. Save additional details to JSON
8. Log success
9. Display confirmation to user

Search Student Flow:

1. Get search term from user
2. Load all students from CSV
3. Loop through students looking for matches in name or reg number
4. Display results
5. Log search action

---

4. EXCEPTION HANDLING STRATEGY

4.1 Types of Exceptions Handled

Exception Type	When Raised	How Handled
StudentNotFoundError	Student doesn't exist	Display error message, continue
InvalidStudentDataError	Validation fails	Display validation error, prompt again
DuplicateStudentError	Student already exists	Display error, prevent duplicate
FileNotFoundError	CSV/JSON file doesn't exist	Create new file automatically
json.JSONDecodeError	Corrupted JSON	Display error, load as empty
ValueError	Invalid data type	Convert or display error
KeyboardInterrupt	User presses Ctrl+C	Graceful shutdown

4.2 Custom Exceptions

python

class StudentNotFoundError(Exception):

"""Raised when a student is not found."""

```
pass


class InvalidStudentDataError(Exception):
"""Raised when student data is invalid."""
pass


class DuplicateStudentError(Exception):
"""Raised when a student with same registration number exists."""
pass
```

4.3 Try/Except/Finally Structure

```
python
try:
# Code that might raise an exception
student = find_student_by_reg(reg_number)

except StudentNotFoundError as e:
# Handle specific exception
print(f"Error: {e}")
logging.warning(f"Student not found: {e}")

except InvalidStudentDataError as e:
# Handle validation errors
print(f"Validation Error: {e}")
logging.warning(f"Validation error: {e}")

except Exception as e:
# Handle any other unexpected errors
print(f"Error: {e}")
logging.error(f"Unexpected error: {e}")

finally:
```

Always execute (cleanup or log completion)

logging.info("Operation completed")

4.4 Logging Strategy

Log Level	When Used	Example
INFO	Normal operations	"Student added: S100000001"
WARNING	Issues that don't stop execution	"Invalid email format"
ERROR	Serious issues	"Error saving students: Permission denied"

5. TESTING RESULTS

5.1 Test Cases

Test ID	Test Case	Expected Result	Actual Result	Status
TC-01	Add student with valid data	Success	Success	PASS
TC-02	Add student with invalid email	Error message	Error: Invalid email format	PASS
TC-03	Add student with duplicate reg number	DuplicateStudentError	Duplicate error	PASS
TC-04	Search for existing student	Display student	Student found	PASS
TC-05	Search for non-existent student	StudentNotFoundError	Not found	PASS
TC-06	Update student details	Success	Updated	PASS

Test ID	Test Case	Expected Result	Actual Result	Status
TC-07	Delete student with confirmation	Success	Deleted	PASS
TC-08	Delete student without confirmation	Cancelled	Cancelled	PASS
TC-09	Load with missing CSV file	Auto-create	Created	PASS
TC-10	View students when empty	Display "No students"	Message shown	PASS

5.2 Test Environment

Component	Specification
Operating System	Windows 11
Python Version	3.12
IDE	VS Code
Test Data	Sample CSV and JSON files

---

6. SAMPLE OUTPUT

6.1 Main Menu

STUDENT RECORD MANAGEMENT SYSTEM

1. Add Student

2. View All Students

3. View Student Details

4. Search Students

5. Update Student

6. Delete Student

7. View System Log

8. Exit

Enter your choice (1-8):

6.2 Add Student Output

ADD NEW STUDENT

Enter student details:

Registration Number (auto-generated): S100000006

Student Name: Peter Okello

Program: Information Technology

Year: 2

Enter additional details (press Enter to skip):

Address: Lira, Uganda

Contact Number: +256-777-123456

Email: peter@example.com

Student added successfully!

Registration Number: S100000006

6.3 View All Students Output

ALL STUDENTS

Total Students: 5

Reg Number	Name	Program	Year	Contact
S100000001	KiraboEria	ComputerScience	3	+256-701-234567
S100000002	SarahSmith	DataScience	2	+256-772-123456
S100000003	JohnDoe	SoftwareEngineering	1	+256-755-987654
S100000004	AliceJohnson	Cybersecurity	4	+256-789-456123
S100000005	RobertMugabe	BusinessComputing	2	+256-701-789012

6.4 Search Output

SEARCH STUDENTS

Enter name or registration number to search: Peter

Found 1 student(s):

Reg Number	Name	Program	Year
S100000006	Peter Okello	Information Technology	2

6.5 Log File Sample

2026-06-28 10:30:00 - INFO - STUDENT SYSTEM STARTED

2026-06-28 10:30:15 - INFO - Add student operation started

2026-06-28 10:30:16 - INFO - Student added: S100000001 - Kirabo Eria

2026-06-28 10:30:16 - INFO - Add student operation completed

2026-06-28 10:40:00 - INFO - View all students operation started

2026-06-28 10:40:00 - INFO - Displayed 5 students

2026-06-28 11:00:00 - INFO - Search students operation started

2026-06-28 11:00:00 - INFO - Search completed. Found 1 results

2026-06-28 11:15:00 - INFO - Update student operation started

2026-06-28 11:15:00 - INFO - Student updated: S100000002

2026-06-28 11:30:00 - INFO - Delete student operation started

2026-06-28 11:30:00 - INFO - Student deleted: S100000003

2026-06-28 12:00:00 - INFO - SYSTEM EXITED

7. CONCLUSION

7.1 Skills Demonstrated

Skill	Implementation
File I/O	Reading/writing CSV and JSON files
Exception Handling	try/except/finally with custom exceptions
Logging	All actions logged with timestamps
Input Validation	Regex patterns and custom validation
Menu-Driven Design	User-friendly interactive system
Modular Programming	Separate functions for each operation
Data Persistence	Data saved between program runs

7.2 System Features

Feature	Status
Add student	Working

Feature	Status
View all students	Working
View student details	Working
Search students	Working
Update student	Working
Delete student	Working
View system log	Working
Error handling	Working
Input validation	Working
Auto-generate registration number	Working

The system is robust, user-friendly, and handles errors gracefully. All data is persisted between sessions, and every action is logged for auditing purposes.

---

8. SUBMISSION CHECKLIST

File	Status
student_management.py (Python source code)	Complete
students.csv (Sample data)	Complete
students_details.json (Sample data)	Complete
student_system.log (Log file)	Complete
This report (1-2 pages)	Complete

REPORT COMPLETE